

Wydział Informatyki Politechniki Białostockiej Przetwarzanie Sygnałów i Obrazów – Pracownia specjalistyczna

Ćwiczenie Nr. 4

Temat: Analiza widmowa sygnałów.

Imię i nazwisko studenta: Michał Pączkowski.

Data realizacji ćwiczenia : 20.04.2026

Wszystkie zdjęcia w: SprawozdaniePsIO4 na github.

Link do repozytorium:

https://github.com/ErythrinaCremesinus/PsIO_Sprawko4

Zadanie 4.1

Interpretacja:

- szum gaussowski: obwiednia powinna być w przybliżeniu stała, bez wyraźnego trendu
- sinusoida 1 kHz: obwiednia szybko zbiega do stałego poziomu
- chirp: obwiednia zmienia się w czasie, bo własności sygnału zależą od chwili")
- sygnał mowy: obwiednia ma wyraźne odcinki o różnej energii, więc sygnał jest niestacjonarny")

Kod:

```
from future import annotations
```

```
from pathlib import Path
```

```

import numpy as np
import matplotlib.pyplot as plt

FS = 8000
DT = 1.0 / FS

def recursive_power_envelope(signal: np.ndarray, alpha: float)
-> np.ndarray:
    power = np.empty_like(signal, dtype=float)
    power[0] = signal[0] ** 2
    for index in range(1, len(signal)):
        power[index] = alpha * power[index - 1] + (1.0 -
alpha) * signal[index] ** 2
    return power

def time_axis(duration: float) -> np.ndarray:
    return np.arange(int(duration * FS)) / FS

def generate_gaussian_noise(duration: float = 3.0) ->
tuple[np.ndarray, np.ndarray]:
    rng = np.random.default_rng(42)
    t = time_axis(duration)
    signal = rng.normal(0.0, 1.0, size=t.shape)
    return t, signal

def generate_sine(duration: float = 3.0, frequency: float =
1000.0) -> tuple[np.ndarray, np.ndarray]:
    t = time_axis(duration)
    signal = np.sin(2.0 * np.pi * frequency * t)
    return t, signal

```

```
def generate_chirp(duration: float = 5.0, f0: float = 0.0, f1:
float = 1000.0) -> tuple[np.ndarray, np.ndarray]:
```

```
    t = time_axis(duration)
```

```
    k = (f1 - f0) / duration
```

```
    phase = 2.0 * np.pi * (f0 * t + 0.5 * k * t**2)
```

```
    signal = np.sin(phase)
```

```
    return t, signal
```

```
def generate_speech_like(duration: float = 3.0) ->
tuple[np.ndarray, np.ndarray]:
```

```
    t = time_axis(duration)
```

```
    signal = np.zeros_like(t)
```

```
    segments = [
```

```
        (0.0, 0.8, 120.0, 0.20, 700.0),
```

```
        (0.8, 1.6, 140.0, 0.30, 900.0),
```

```
        (1.6, 2.1, None, 0.12, None),
```

```
        (2.1, 3.0, 110.0, 0.25, 500.0),
```

```
    ]
```

```
    for start, end, pitch, amplitude, formant in segments:
```

```
        mask = (t >= start) & (t < end)
```

```
        local_t = t[mask] - start
```

```
        if pitch is None:
```

```
            rng = np.random.default_rng(7)
```

```
            noise = rng.normal(0.0, 1.0, size=local_t.shape)
```

```
            envelope = 0.7 + 0.3 * np.sin(2.0 * np.pi * 3.0 *
local_t) ** 2
```

```
            signal[mask] = amplitude * envelope * noise
```

```

        continue

    glottal = (
        np.sin(2.0 * np.pi * pitch * local_t)
        + 0.5 * np.sin(2.0 * np.pi * 2.0 * pitch *
local_t)
        + 0.25 * np.sin(2.0 * np.pi * 3.0 * pitch *
local_t)
    )

    modulator = 0.6 + 0.4 * np.sin(2.0 * np.pi * 1.5 *
local_t) ** 2

    if formant is not None:
        resonance = 1.0 + 0.4 * np.sin(2.0 * np.pi *
formant * local_t / FS)
    else:
        resonance = 1.0

    signal[mask] = amplitude * modulator * resonance *
glottal

    return t, signal

def load_or_generate_speech() -> tuple[np.ndarray,
np.ndarray]:
    script_dir = Path(__file__).resolve().parent
    candidates = sorted(script_dir.glob("*.wav"))
    if candidates:
        try:
            from scipy.io import wavfile

            sample_rate, data = wavfile.read(candidates[0])
            if data.ndim > 1:

```

```

        data = data[:, 0]
        data = data.astype(float)
        if np.max(np.abs(data)) > 0:
            data /= np.max(np.abs(data))
        if sample_rate != FS:
            duration = len(data) / sample_rate
            old_t = np.linspace(0.0, duration, len(data),
endpoint=False)
            new_t = time_axis(duration)
            data = np.interp(new_t, old_t, data)
            return new_t, data
        return time_axis(len(data) / sample_rate), data
    except Exception:
        pass

```

```

return generate_speech_like()

```

```

def plot_signal_with_envelopes(ax, t: np.ndarray, signal:
np.ndarray, title: str, alphas: list[float]) -> None:

```

```

    ax.plot(t, signal, color="0.7", linewidth=0.8,
label="sygnał")

```

```

    for alpha in alphas:
        envelope = recursive_power_envelope(signal, alpha)
        ax.plot(t, envelope, linewidth=1.4, label=f"P[n],
α={alpha}")
        ax.set_title(title)
        ax.set_xlabel("czas [s]")
        ax.set_ylabel("amplituda / moc")
        ax.grid(True, alpha=0.25)

```

```

def main() -> None:

```

```

alpha_values = [0.9, 0.99, 0.999]

signals = [
    ("Szum gaussowski", *generate_gaussian_noise(3.0)),
    ("Sinusoida 1 kHz", *generate_sine(3.0, 1000.0)),
    ("Chirp 0 Hz -> 1 kHz", *generate_chirp(5.0, 0.0,
1000.0)),
    ("Sygnał mowy", *load_or_generate_speech()),
]

fig, axes = plt.subplots(2, 2, figsize=(15, 9),
constrained_layout=True)

axes = axes.ravel()

for ax, (title, t, signal) in zip(axes, signals):
    plot_signal_with_envelopes(ax, t, signal, title,
alpha_values)
    ax.legend(loc="upper right", fontsize=9)

fig.suptitle("Zadanie 4.1 - obwiednia mocy liczona
rekurencyjnie", fontsize=16)

backend = plt.get_backend().lower()

if "agg" in backend or "inline" in backend:
    output_path =
Path(__file__).with_name("zad4_1_wykresy.png")
    fig.savefig(output_path, dpi=200)
    print(f"Zapisano wykres do: {output_path}")
else:
    plt.show()

if __name__ == "__main__":

```

```
main()
```

Zadanie 4.2

Kod:

```
from future import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

FS = 8000
N = 1024
F1 = 1000.0
F2 = 2000.0

def generate_signals() -> dict[str, np.ndarray]:
    rng = np.random.default_rng(12345)
    n = np.arange(N)
    w = rng.normal(0.0, 1.0, size=N)
    s = 0.5 * np.sin(2.0 * np.pi * F1 * n / FS) + np.sin(2.0 *
np.pi * F2 * n / FS)
    y = s + 0.1 * w
    return {"w[n]": w, "s[n]": s, "y[n]": y}

def safe_db(values: np.ndarray, reference: float = 1.0) ->
np.ndarray:
```

```
    return 10.0 * np.log10(np.maximum(values, 1e-15) /  
reference)
```

```
def one_sided_frequency_axis(length: int, fs: float = FS) ->  
np.ndarray:
```

```
    return np.fft.rfftfreq(length, d=1.0 / fs)
```

```
def periodogram(signal: np.ndarray, fs: float = FS) ->  
tuple[np.ndarray, np.ndarray]:
```

```
    length = len(signal)
```

```
    spectrum = np.fft.rfft(signal)
```

```
    power = (np.abs(spectrum) ** 2) / length
```

```
    if power.size > 2:
```

```
        power[1:-1] *= 2.0
```

```
    return one_sided_frequency_axis(length, fs), power
```

```
def fft_power_spectrum(signal: np.ndarray, fs: float = FS,  
nfft: int | None = None) -> tuple[np.ndarray, np.ndarray]:
```

```
    if nfft is None:
```

```
        nfft = len(signal)
```

```
    spectrum = np.fft.rfft(signal, n=nfft)
```

```
    power = (np.abs(spectrum) ** 2) / len(signal)
```

```
    if power.size > 2:
```

```
        power[1:-1] *= 2.0
```

```
    return one_sided_frequency_axis(nfft, fs), power
```

```
def hann_window_spectrum(signal: np.ndarray, window_length:  
int, fs: float = FS) -> tuple[np.ndarray, np.ndarray]:
```

```
    window = np.hanning(window_length)
```

```
    windowed = signal[:window_length] * window
```

```
    coherent_gain = np.sum(window) / window_length
```



```

    if coherent_gain == 0:
        coherent_gain = 1.0

    spectrum = np.fft.rfft(windowed, n=window_length)

    power = (np.abs(spectrum) ** 2) / (window_length *
coherent_gain**2)

    if power.size > 2:
        power[1:-1] *= 2.0

    return one_sided_frequency_axis(window_length, fs), power


def plot_spectrum(ax, freqs: np.ndarray, power: np.ndarray,
label: str, style: str = "-", linewidth: float = 1.2) -> None:

    ax.plot(freqs, safe_db(power), style, linewidth=linewidth,
label=label)

    ax.set_xlim(0, FS / 2)

    ax.set_xlabel("częstotliwość [Hz]")

    ax.set_ylabel("poziom [dB]")

    ax.grid(True, alpha=0.25)


def analyze_signal(signal_name: str, signal: np.ndarray) ->
tuple[plt.Figure, plt.Figure]:

    fig, axes = plt.subplots(2, 2, figsize=(15, 9),
constrained_layout=True)

    axes = axes.ravel()

    freqs_p, power_p = periodogram(signal)

    plot_spectrum(axes[0], freqs_p, power_p, "periodogram")

    axes[0].set_title(f"{signal_name} - periodogram")

    axes[0].legend(loc="upper right")

    freqs_f, power_f = fft_power_spectrum(signal)

```

```

    plot_spectrum(axes[1], freqs_f, power_f, "FFT",
linewidth=1.0)

    axes[1].set_title(f"{signal_name} - FFT")
    axes[1].legend(loc="upper right")

    for ax, window_length in zip(axes[2:], [256, 128]):
        freqs_w, power_w = hann_window_spectrum(signal,
window_length)

        plot_spectrum(ax, freqs_w, power_w, f"Hann
{window_length}")

        ax.set_title(f"{signal_name} - FFT z oknem Hanna
{window_length}")

        ax.legend(loc="upper right")

    extra_fig, extra_ax = plt.subplots(figsize=(8, 4.8),
constrained_layout=True)

    freqs_w64, power_w64 = hann_window_spectrum(signal, 64)
    plot_spectrum(extra_ax, freqs_w64, power_w64, "Hann 64")
    extra_ax.set_title(f"{signal_name} - FFT z oknem Hanna
64")

    extra_ax.legend(loc="upper right")

    fig.suptitle(f"Zadanie 4.2 - analiza widmowa:
{signal_name}", fontsize=16)

    extra_fig.suptitle(f"Zadanie 4.2 - analiza widmowa:
{signal_name} (okno 64)", fontsize=16)

    return fig, extra_fig

def save_or_show(figures: list[tuple[str, plt.Figure]]) ->
None:

    backend = plt.get_backend().lower()

    if "agg" in backend or "inline" in backend:

```

```

    output_dir = Path(__file__).resolve().parent
    for name, fig in figures:
        output_path = output_dir / f"zad4_2_{name}.png"
        fig.savefig(output_path, dpi=200)
        print(f"Zapisano wykres: {output_path}")
    else:
        plt.show()

def main() -> None:
    signals = generate_signals()
    figures: list[tuple[str, plt.Figure]] = []

    for name, signal in signals.items():
        fig_main, fig_64 = analyze_signal(name, signal)
        safe_name = name.replace("[", "").replace("]",
        "").replace(" ", "_")
        figures.append((f"{safe_name}_main", fig_main))
        figures.append((f"{safe_name}_hann64", fig_64))

    save_or_show(figures)

if __name__ == "__main__":
    main()

```

Wnioski:

- $w[n]$ ma widmo szerokopasmowe, bez wyraźnych linii widmowych
- $s[n]$ ma dwa dominujące piki przy 1 kHz i 2 kHz
- $y[n]$ zachowuje piki $s[n]$, ale ma podniesione tło szumowe
- Okno Hanna zmniejsza przecieki widmowe; krótsze okna dają gorszą rozdzielczość częstotliwościową

Zadanie 4.3

Kod:

```
from future import annotations

import argparse
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

try:
    from scipy.io import wavfile
    from scipy.signal import spectrogram
except Exception as exc: # pragma: no cover - depends on
    environment
    raise RuntimeError(
        "Do uruchomienia tego skryptu potrzebne są pakiety
        scipy i matplotlib."
    ) from exc

WINDOW_LENGTH = 1024
OVERLAP = 768
DEFAULT_FALLBACK_FS = 8000

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Zadanie 4.3
    - analiza sygnału")
    parser.add_argument(
```

```

        "input",
        nargs="?",
        default=None,
        help="Ścieżka do pliku WAV. Jeśli brak, skrypt użyje
pierwszego WAV z katalogu lub sygnału demonstracyjnego.",
    )

    return parser.parse_args()

def normalize_signal(signal: np.ndarray) -> np.ndarray:
    signal = signal.astype(float)
    peak = np.max(np.abs(signal))
    if peak > 0:
        signal = signal / peak
    return signal

def generate_demo_signal(fs: int = DEFAULT_FALLBACK_FS,
duration: float = 3.0) -> tuple[int, np.ndarray]:
    t = np.arange(int(fs * duration)) / fs
    signal = np.zeros_like(t)

    first = t < 1.0
    second = (t >= 1.0) & (t < 2.0)
    third = t >= 2.0

    signal[first] = 0.9 * np.sin(2.0 * np.pi * 200.0 *
t[first])

    signal[second] = 0.6 * np.sin(2.0 * np.pi * (400.0 *
t[second] + 200.0 * t[second] ** 2))

    rng = np.random.default_rng(7)

    signal[third] = 0.15 * rng.normal(0.0, 1.0,
size=np.count_nonzero(third))

```

```

    return fs, signal

def find_default_wav(script_dir: Path) -> Path | None:
    preferred = script_dir / "4.3.wav"
    if preferred.exists():
        return preferred

    candidates = sorted(script_dir.glob("*.wav"))
    return candidates[0] if candidates else None

def load_signal(input_path: str | None, script_dir: Path) ->
tuple[int, np.ndarray, str]:
    if input_path:
        path = Path(input_path)
    else:
        path = find_default_wav(script_dir)

    if path is None:
        fs, signal = generate_demo_signal()
        return fs, signal, "sygnał demonstracyjny"

    fs, data = wavfile.read(path)
    if data.ndim > 1:
        data = np.mean(data, axis=1)
    signal = normalize_signal(data)
    return int(fs), signal, str(path)

def safe_db(values: np.ndarray, reference: float = 1.0) ->
np.ndarray:
```

```
    return 10.0 * np.log10(np.maximum(values, 1e-15) /  
reference)
```

```
def one_sided_fft(signal: np.ndarray, fs: int) ->  
tuple[np.ndarray, np.ndarray]:  
    spectrum = np.fft.rfft(signal)  
    freqs = np.fft.rfftfreq(len(signal), d=1.0 / fs)  
    amplitude = np.abs(spectrum) / len(signal)  
    if amplitude.size > 2:  
        amplitude[1:-1] *= 2.0  
    return freqs, amplitude
```

```
def compute_spectrogram(signal: np.ndarray, fs: int) ->  
tuple[np.ndarray, np.ndarray, np.ndarray]:  
    freqs, times, sxx = spectrogram(  
        signal,  
        fs=fs,  
        window="hamming",  
        nperseg=WINDOW_LENGTH,  
        noverlap=OVERLAP,  
        nfft=WINDOW_LENGTH,  
        scaling="density",  
        mode="magnitude",  
    )  
    power = sxx**2  
    return freqs, times, power
```

```
def plot_time_domain(ax, signal: np.ndarray, fs: int, title:  
str) -> None:  
    time = np.arange(len(signal)) / fs  
    ax.plot(time, signal, color="#1f77b4", linewidth=1.0)
```

```

    ax.set_title(title)

    ax.set_xlabel("czas [s]")

    ax.set_ylabel("amplituda")

    ax.grid(True, alpha=0.25)

def plot_fft(ax, signal: np.ndarray, fs: int, title: str) ->
None:

    freqs, amplitude = one_sided_fft(signal, fs)

    ax.plot(freqs, safe_db(amplitude), color="#d62728",
linewidth=1.0)

    ax.set_title(title)

    ax.set_xlabel("częstotliwość [Hz]")

    ax.set_ylabel("amplituda [dB]")

    ax.grid(True, alpha=0.25)

    ax.set_xlim(0, fs / 2)

def plot_spectrogram(ax, signal: np.ndarray, fs: int, title:
str) -> None:

    freqs, times, power = compute_spectrogram(signal, fs)

    pcm = ax.pcolormesh(times, freqs, safe_db(power),
shading="auto", cmap="magma")

    ax.set_title(title)

    ax.set_xlabel("czas [s]")

    ax.set_ylabel("częstotliwość [Hz]")

    ax.set_ylim(0, fs / 2)

    return pcm

def save_or_show(figures: list[tuple[str, plt.Figure]]) ->
None:

    for name, fig in figures:

```



```

        output =
Path(__file__).with_name(f"zad4_3_{name}.png")
        fig.savefig(output, dpi=200)
        print(f"Zapisano wykres: {output}")

backend = plt.get_backend().lower()
if "agg" not in backend and "inline" not in backend:
    plt.show()

def create_analysis_figure(signal: np.ndarray, fs: int,
source_name: str) -> plt.Figure:
    fig, axes = plt.subplots(3, 1, figsize=(15, 11),
constrained_layout=True)

    plot_time_domain(axes[0], signal, fs, f"{source_name} -
dziedzina czasu")

    plot_fft(axes[1], signal, fs, f"{source_name} - FFT")

    pcm = plot_spectrogram(axes[2], signal, fs,
f"{source_name} - spektrogram Hamming {WINDOW_LENGTH}")

    fig.colorbar(pcm, ax=axes[2], label="poziom mocy [dB]")

    fig.suptitle("Zadanie 4.3 - analiza sygnału w czasie i
częstotliwości", fontsize=16)

    return fig

def main() -> None:
    args = parse_args()

    script_dir = Path(__file__).resolve().parent

    fs, signal, source_name = load_signal(args.input,
script_dir)

    fig = create_analysis_figure(signal, fs, source_name)

```

```
    save_or_show([(Path(source_name).stem.replace(".",
"_").replace(" ", "_"), fig)])
```

```
    duration = len(signal) / fs
```

```
if __name__ == "__main__":
    main()
```

Wnioski:

- Sygnał ma długość {duration:.3f} s i jest próbkowany z $fs = \{fs\}$ Hz
- FFT pokazuje globalny rozkład energii w całym sygnale
- Spektrogram pokazuje, jak energia zmienia się w czasie i częstotliwości
- Dla modułu STFT i widma mocy w dB obowiązuje zależność $10\log_{10}(|X|^2) = 20\log_{10}(|X|)$
- Jeśli sygnał jest mowy lub modulowany, energia zwykle skupia się w pasmach formantowych i w krótkich odcinkach czasu

Zadanie 4.4

Kod:

```
from future import annotations
```

```
from pathlib import Path
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
try:
```

```
    from scipy.io import wavfile
```

```
    from scipy.signal import spectrogram as scipy_spectrogram
```

```
except Exception as exc: # pragma: no cover - environment
dependent
```

```
    raise RuntimeError("Do uruchomienia wymagane są pakiety
scipy i matplotlib.") from exc
```

```
WINDOW_LENGTH = 1024
```

```
OVERLAP = 768
```

```
HOP = WINDOW_LENGTH - OVERLAP
```

```
DEFAULT_FALLBACK_FS = 8000
```

```
def normalize_signal(signal: np.ndarray) -> np.ndarray:
```

```
    signal = signal.astype(float)
```

```
    peak = np.max(np.abs(signal))
```

```
    if peak > 0:
```

```
        signal = signal / peak
```

```
    return signal
```

```
def generate_demo_signal(fs: int = DEFAULT_FALLBACK_FS,
duration: float = 3.0) -> tuple[int, np.ndarray]:
```

```
    t = np.arange(int(fs * duration)) / fs
```

```
    signal = np.zeros_like(t)
```

```
    first = t < 1.0
```

```
    second = (t >= 1.0) & (t < 2.0)
```

```
    third = t >= 2.0
```

```
    signal[first] = 0.9 * np.sin(2.0 * np.pi * 220.0 *
t[first])
```

```
    signal[second] = 0.6 * np.sin(2.0 * np.pi * (300.0 *
t[second] + 250.0 * t[second] ** 2))
```

```
    rng = np.random.default_rng(11)
```

```

    signal[third] = 0.12 * rng.normal(0.0, 1.0,
size=np.count_nonzero(third))

    return fs, signal

def find_default_wav(script_dir: Path) -> Path | None:
    candidates = sorted(script_dir.glob("*.wav"))
    return candidates[0] if candidates else None

def load_signal(script_dir: Path) -> tuple[int, np.ndarray,
str]:
    path = find_default_wav(script_dir)
    if path is None:
        fs, signal = generate_demo_signal()
        return fs, signal, "sygnał demonstracyjny"

    fs, data = wavfile.read(path)
    if data.ndim > 1:
        data = np.mean(data, axis=1)
    signal = normalize_signal(data)
    return int(fs), signal, str(path)

def safe_db(values: np.ndarray) -> np.ndarray:
    return 10.0 * np.log10(np.maximum(values, 1e-15))

def one_sided_fft(signal: np.ndarray, fs: int) ->
tuple[np.ndarray, np.ndarray]:
    spectrum = np.fft.rfft(signal)
    freqs = np.fft.rfftfreq(len(signal), d=1.0 / fs)
    amplitude = np.abs(spectrum) / len(signal)

```

```

    if amplitude.size > 2:
        amplitude[1:-1] *= 2.0
    return freqs, amplitude

def custom_smoothed_periodogram_spectrogram(
    signal: np.ndarray,
    fs: int,
    window_length: int = WINDOW_LENGTH,
    overlap: int = OVERLAP,
    time_smoothing: int = 3,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    hop = window_length - overlap
    if hop <= 0:
        raise ValueError("overlap musi być mniejszy od
window_length")

    window = np.hamming(window_length)
    window_energy = np.sum(window**2)
    if window_energy == 0:
        window_energy = 1.0

    if len(signal) < window_length:
        signal = np.pad(signal, (0, window_length -
len(signal)))

    starts = np.arange(0, len(signal) - window_length + 1,
hop)
    if starts.size == 0:
        starts = np.array([0])

```

```

spectrum_columns = []
times = []

for start in starts:
    frame = signal[start : start + window_length]
    if len(frame) < window_length:
        frame = np.pad(frame, (0, window_length -
len(frame)))

    windowed = frame * window
    spectrum = np.fft.rfft(windowed, n=window_length)
    periodogram = (np.abs(spectrum) ** 2) / (fs *
window_energy)

    if periodogram.size > 2:
        periodogram[1:-1] *= 2.0
    spectrum_columns.append(periodogram)
    times.append((start + window_length / 2.0) / fs)

power = np.stack(spectrum_columns, axis=1)

if time_smoothing > 1:
    kernel = np.ones(time_smoothing) / time_smoothing
    smoothed = np.empty_like(power)
    for bin_index in range(power.shape[0]):
        smoothed[bin_index, :] =
np.convolve(power[bin_index, :], kernel, mode="same")
    power = smoothed

freqs = np.fft.rfftfreq(window_length, d=1.0 / fs)
return freqs, np.asarray(times), power

```

```

def reference_spectrogram(signal: np.ndarray, fs: int) ->
tuple[np.ndarray, np.ndarray, np.ndarray]:
    freqs, times, sxx = scipy_spectrogram(
        signal,
        fs=fs,
        window="hamming",
        nperseg=WINDOW_LENGTH,
        noverlap=OVERLAP,
        nfft=WINDOW_LENGTH,
        scaling="density",
        mode="psd",
    )
    return freqs, times, sxx

def plot_time_domain(ax, signal: np.ndarray, fs: int, title:
str) -> None:
    time = np.arange(len(signal)) / fs
    ax.plot(time, signal, color="#1f77b4", linewidth=0.9)
    ax.set_title(title)
    ax.set_xlabel("czas [s]")
    ax.set_ylabel("amplituda")
    ax.grid(True, alpha=0.25)

def plot_fft(ax, signal: np.ndarray, fs: int, title: str) ->
None:
    freqs, amplitude = one_sided_fft(signal, fs)
    ax.plot(freqs, safe_db(amplitude), color="#d62728",
linewidth=0.9)
    ax.set_title(title)
    ax.set_xlabel("częstotliwość [Hz]")
    ax.set_ylabel("amplituda [dB]")

```

```

ax.set_xlim(0, fs / 2)

ax.grid(True, alpha=0.25)

def plot_3d_mesh(ax, freqs: np.ndarray, times: np.ndarray,
power: np.ndarray, title: str) -> None:

    time_grid, freq_grid = np.meshgrid(times, freqs)

    surface = ax.plot_surface(

        time_grid,

        freq_grid,

        safe_db(power),

        cmap="magma",

        linewidth=0,

        antialiased=True,

        rcount=min(80, power.shape[0]),

        ccount=min(80, power.shape[1]),

    )

    ax.set_title(title)

    ax.set_xlabel("czas [s]")

    ax.set_ylabel("częstotliwość [Hz]")

    ax.set_zlabel("gęstość mocy [dB]")

    ax.set_ylim(0, freqs.max())

    return surface


def save_figures(figures: list[tuple[str, plt.Figure]]) ->
None:

    for name, fig in figures:

        output =
Path(__file__).with_name(f"zad4_4_{name}.png")

        fig.savefig(output, dpi=200)

        print(f"Zapisano wykres: {output}")

```



```

def create_main_figure(signal: np.ndarray, fs: int,
source_name: str) -> plt.Figure:

    fig = plt.figure(figsize=(16, 8), constrained_layout=True)

    grid = fig.add_gridspec(2, 1)

    ax_time = fig.add_subplot(grid[0, 0])
    ax_fft = fig.add_subplot(grid[1, 0])

    plot_time_domain(ax_time, signal, fs, f"{source_name} -
dziedzina czasu")

    plot_fft(ax_fft, signal, fs, f"{source_name} - FFT")

    fig.suptitle("Zadanie 4.4 - sygnał i FFT", fontsize=16)

    return fig

```

```

def create_3d_figure(signal: np.ndarray, fs: int, source_name:
str) -> plt.Figure:

    fig = plt.figure(figsize=(16, 9), constrained_layout=True)

    ax_mesh = fig.add_subplot(111, projection="3d")

    freqs, times, power =
custom_smoothed_periodogram_spectrogram(signal, fs)

    surface = plot_3d_mesh(ax_mesh, freqs, times, power,
f"{source_name} - własny spektrogram 3D")

    fig.colorbar(surface, ax=ax_mesh, shrink=0.8, pad=0.1,
label="gęstość mocy [dB]")

    fig.suptitle("Zadanie 4.4 - własny spektrogram 3D",
fontsize=16)

    return fig

```

```

def create_comparison_figure(signal: np.ndarray, fs: int,
source_name: str) -> plt.Figure:

```

```

    custom_freqs, custom_times, custom_power =
custom_smoothed_periodogram_spectrogram(signal, fs)

    ref_freqs, ref_times, ref_power =
reference_spectrogram(signal, fs)

    fig, axes = plt.subplots(1, 2, figsize=(16, 6),
constrained_layout=True)

    mesh0 = axes[0].pcolormesh(custom_times, custom_freqs,
safe_db(custom_power), shading="auto", cmap="magma")

    axes[0].set_title("Własny spektrogram")
    axes[0].set_xlabel("czas [s]")
    axes[0].set_ylabel("częstotliwość [Hz]")
    axes[0].set_ylim(0, fs / 2)

    fig.colorbar(mesh0, ax=axes[0], label="gęstość mocy [dB]")

    mesh1 = axes[1].pcolormesh(ref_times, ref_freqs,
safe_db(ref_power), shading="auto", cmap="magma")

    axes[1].set_title("Referencyjny spektrogram SciPy")
    axes[1].set_xlabel("czas [s]")
    axes[1].set_ylabel("częstotliwość [Hz]")
    axes[1].set_ylim(0, fs / 2)

    fig.colorbar(mesh1, ax=axes[1], label="gęstość mocy [dB]")

    fig.suptitle(f"Zadanie 4.4 - porównanie spektrogramów:
{source_name}", fontsize=16)

    return fig

def main() -> None:

    script_dir = Path(__file__).resolve().parent

    fs, signal, source_name = load_signal(script_dir)

```

```

main_fig = create_main_figure(signal, fs, source_name)
three_d_fig = create_3d_figure(signal, fs, source_name)
comparison_fig = create_comparison_figure(signal, fs,
source_name)

safe_name = Path(source_name).stem.replace(".",
"_").replace(" ", "_")

save_figures([
    (f"{safe_name}_main", main_fig),
    (f"{safe_name}_3d", three_d_fig),
    (f"{safe_name}_compare", comparison_fig),
])

plt.show()

duration = len(signal) / fs

if __name__ == "__main__":
    main()

```

Wnioski:

- Sygnał ma długość {duration:.3f} s i jest próbkowany z $fs = \{fs\}$ Hz
- FFT pokazuje globalny rozkład energii, a spektrogram pokazuje jego zmianę w czasie
- Własny spektrogram oparty na periodogramach powinien być bardzo zbliżony do referencji SciPy
- Wygładzanie po czasie zmniejsza ziarnistość obrazu kosztem rozdzielczości czasowej
- Wynik w skali dB dobrze uwidacznia lokalne maksima energii i tło szumowe

Zadanie 4.5

Kod:

```

from future import annotations

```

```

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

try:
    from noise import pnoise2
    _USE_LIBRARY = True
except ModuleNotFoundError:
    _USE_LIBRARY = False

def _fade(t: np.ndarray) -> np.ndarray:
    return t * t * t * (t * (t * 6.0 - 15.0) + 10.0)

def _lerp(a: np.ndarray, b: np.ndarray, t: np.ndarray) ->
np.ndarray:
    return a + t * (b - a)

def _build_permutation(seed: int = 0) -> np.ndarray:
    rng = np.random.default_rng(seed)
    p = np.arange(256, dtype=int)
    rng.shuffle(p)
    return np.tile(p, 2)

def _gradient(h: np.ndarray, x: np.ndarray, y: np.ndarray) ->
np.ndarray:
    vectors = np.array([[1, 1], [-1, 1], [1, -1], [-1, -1],
                        [1, 0], [-1, 0], [0, 1], [0, -1]],
dtype=float)

```

```

g = vectors[h % 8]

return g[..., 0] * x + g[..., 1] * y

def _perlin2(x: np.ndarray, y: np.ndarray, perm: np.ndarray) -
> np.ndarray:
    xi = np.floor(x).astype(int) & 255
    yi = np.floor(y).astype(int) & 255
    xf = x - np.floor(x)
    yf = y - np.floor(y)

    u = _fade(xf)
    v = _fade(yf)

    aa = perm[perm[xi    ] + yi    ]
    ab = perm[perm[xi    ] + yi + 1]
    ba = perm[perm[xi + 1] + yi    ]
    bb = perm[perm[xi + 1] + yi + 1]

    x1 = _lerp(_gradient(aa, xf,    yf    ),
               _gradient(ba, xf - 1, yf    ), u)
    x2 = _lerp(_gradient(ab, xf,    yf - 1),
               _gradient(bb, xf - 1, yf - 1), u)
    return _lerp(x1, x2, v)

def generate_noise_map_numpy(
    width: int,
    height: int,
    scale: float = 4.0,
    octaves: int = 6,
    persistence: float = 0.5,

```

```

        lacunarity: float = 2.0,
        seed: int = 42,
) -> np.ndarray:
    """Generuje mapę szumu Perlin przez sumowanie oktafów
    (fBm)."""

    perm = _build_permutation(seed)
    xs = np.linspace(0, scale, width, endpoint=False)
    ys = np.linspace(0, scale, height, endpoint=False)
    xg, yg = np.meshgrid(xs, ys)

    noise_map = np.zeros((height, width))
    amplitude = 1.0
    frequency = 1.0
    max_val = 0.0

    for _ in range(octaves):
        noise_map += amplitude * _perlin2(xg * frequency, yg *
frequency, perm)
        max_val += amplitude
        amplitude *= persistence
        frequency *= lacunarity

    return noise_map / max_val

def generate_noise_map_library(
    width: int,
    height: int,
    scale: float = 4.0,
    octaves: int = 6,
    persistence: float = 0.5,

```

```

    lacunarity: float = 2.0,
    seed: int = 42,
) -> np.ndarray:
    """Generuje mapę szumu Perlin z użyciem pnoise2."""
    noise_map = np.empty((height, width))
    for row in range(height):
        for col in range(width):
            x = col / width * scale
            y = row / height * scale
            noise_map[row, col] = pnoise2(
                x, y,
                octaves=octaves,
                persistence=persistence,
                lacunarity=lacunarity,
                base=seed,
            )
    return noise_map

```

```

def generate_noise_map(
    width: int = 512,
    height: int = 512,
    scale: float = 4.0,
    octaves: int = 6,
    persistence: float = 0.5,
    lacunarity: float = 2.0,
    seed: int = 42,
) -> np.ndarray:
    if _USE_LIBRARY:
        print("Użyto: pnoise2 z biblioteki noise")

```

```

        return generate_noise_map_library(width, height,
scale, octaves, persistence, lacunarity, seed)

    else:

        print("Użyto: własna implementacja Perlin 2D (numpy)")

        return generate_noise_map_numpy(width, height, scale,
octaves, persistence, lacunarity, seed)

```

```

TERRAIN_LEVELS = [
    (-1.00, -0.15, ( 2, 50, 127)),
    (-0.15, 0.00, ( 0, 128, 205)),
    ( 0.00, 0.08, (231, 203, 153)),
    ( 0.08, 0.25, (101, 179, 22)),
    ( 0.25, 0.40, ( 2, 75, 26)),
    ( 0.40, 1.01, (203, 178, 78)),
]

```

```

def colorize(noise_map: np.ndarray) -> np.ndarray:
    height, width = noise_map.shape
    rgb = np.zeros((height, width, 3), dtype=np.uint8)
    for lo, hi, color in TERRAIN_LEVELS:
        mask = (noise_map >= lo) & (noise_map < hi)
        for channel, value in enumerate(color):
            rgb[..., channel][mask] = value
    return rgb

```

```

def main() -> None:

    WIDTH, HEIGHT = 512, 512

    noise_map = generate_noise_map(
        width=WIDTH,

```



```

        height=HEIGHT,
        scale=4.0,
        octaves=6,
        persistence=0.5,
        lacunarity=2.0,
        seed=42,
    )

    rgb = colorize(noise_map)

    fig, ax = plt.subplots(figsize=(8, 8),
                             constrained_layout=True)

    ax.imshow(rgb, interpolation="nearest")
    ax.axis("off")

    fig.suptitle("Zadanie 4.5 - mapa terenu z szumu Perlin
2D", fontsize=14)

    output = Path(__file__).with_name("zad4_5_mapa.png")
    fig.savefig(output, dpi=200)
    print(f"Zapisano: {output}")

    plt.show()

if __name__ == "__main__":
    main()

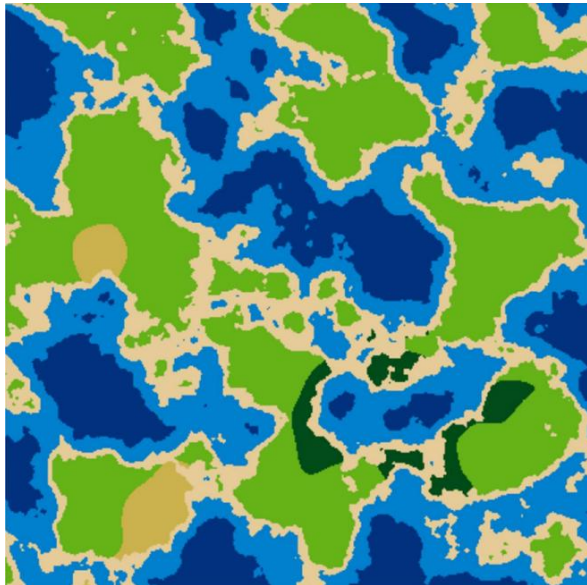
```

Wnioski:

Mimo podjęcia starań w celu wiernego odwzorowania mapy przy użyciu algorytmu szumu (noise), ostateczny rezultat wykazuje pewne odstępstwa od wzorca. Choć paleta barw oraz układ warstw zostały dobrane z dużą precyzją, trudność sprawiło zachowanie identycznej

struktury i zagęszczenia biomów, w szczególności w obszarach leśnych oraz górskich (pustynnych?).

Oryginalna mapa:



Mapa z kodu:

